Processamento de Transações

Raqueline Penteado

Introdução

- Transações
  - Unidade lógica de processamento de banco de dados
- Importante para
  - Reservas aéreas, compras online, sistemas bancários...
- O que os SGBDs devem tratar?
  - Multiusuários
    - Concorrência intercalada
      - Plano de execução
        - Serialidade
      - Recuperação
        - Restaurabilidade

Transação

- Propriedades desejáveis
  - Atomicidade: unidade atômica de processamento.
  - Consistência: após uma transação ser executada o BD deve continuar consistente.
  - Isolamento: uma transação deve ser executada como se estivesse isolada das demais.
  - Durabilidade: todas as alterações provocadas por uma transação deve ser persistida no BD.

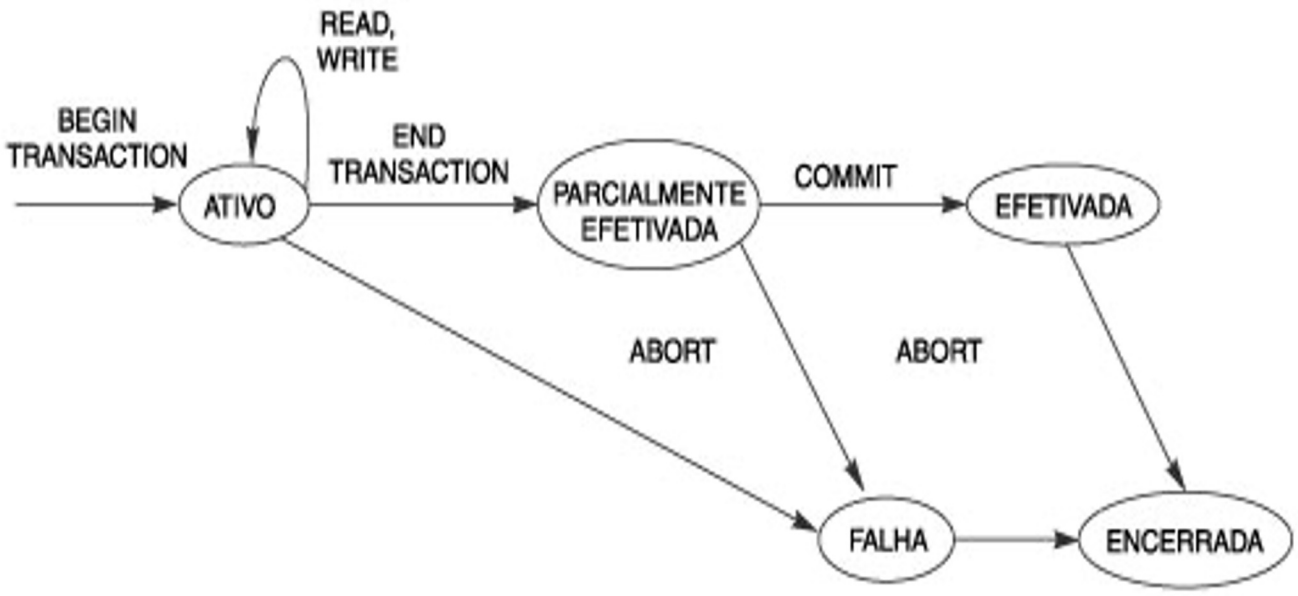
Introdução

- Exemplo de uma transação

```
BEGIN TRANSACTION;
UPDATE CONTA 123 {SALDO := SALDO – 100};
IF ocorreu algum erro THEN
  GO TO UNDO;
END IF;
UPDATE CONTA 456 {SALDO := SALDO + 100};
IF ocorreu algum erro THEN
  GO TO UNDO;
END IF;
COMMIT;
GO TO FINISH;
UNDO:
  ROLLBACK;
FINISH:
  RETURN;
END TRANSACTION;
```

Transação

- Diagrama de transição de estado



Concorrência

- Descontrole da execução das operações
  - Problemas de inconsistência no BD.

- Problemas:
  - Problema da atualização perdida
  - Problema da atualização temporária
  - Problema do sumário incorreto

Concorrência

- Exemplo de duas transações que podem ser concorrentes

(a)

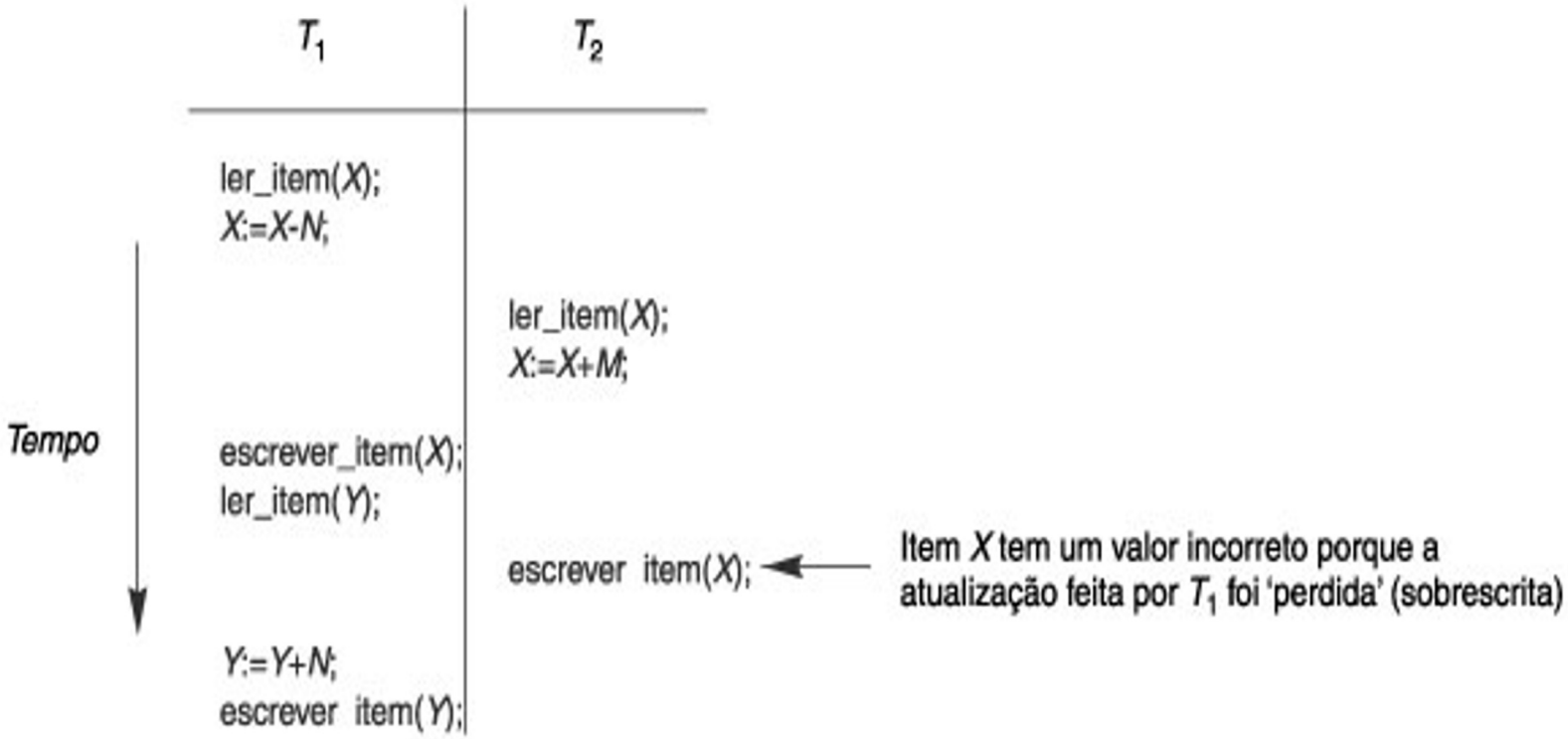
T_1
ler_item (*X*);
X:=*X*-*N*;
escrever_item (*X*);
ler_item (*Y*);
Y:=*Y*+*N*;
escrever_item (*Y*);

(b)

T_2
ler_item (*X*);
X:=*X*+*M*;
escrever_item (*X*);

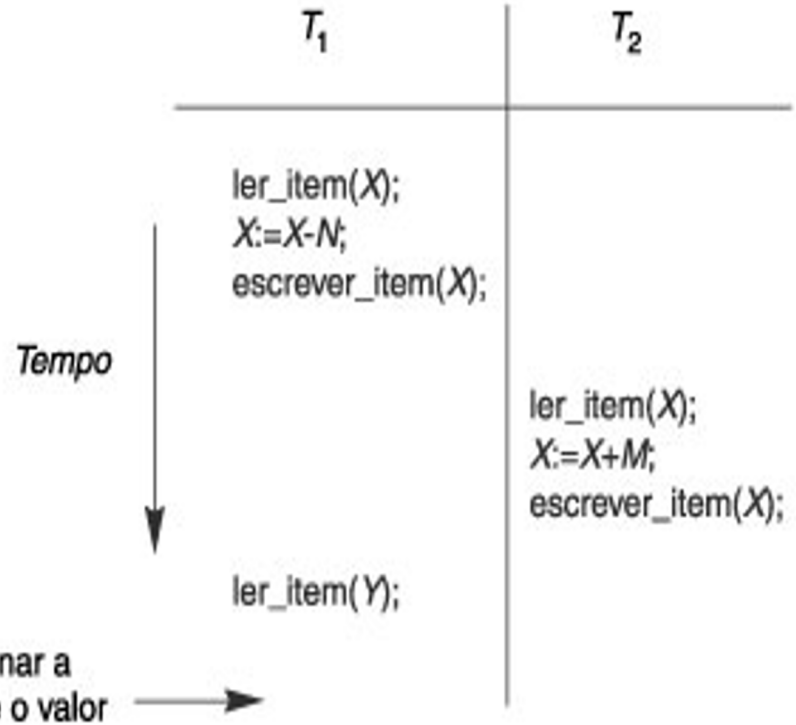
Concorrência

- Problema da atualização perdida
  - O valor final de X será incorreto no final da execução



Concorrência

- Problema da atualização temporária
  - Leitura de sujeira



A transação T_1 falha e X deverá retornar a seu valor original; enquanto isso, T_2 lê o valor 'temporário' incorreto de X.

Concorrência

- Problema do sumário incorreto

T_1	T_3

	sum:=0;
	ler_item(*A*);
	sum:=*sum*+*A*;
	⋮

ler_item(*X*);
X:=*X*-N;
escrever_item(*X*);	

	ler_item(*X*);
	sum:=*sum*+*X*;
	ler_item(*Y*);
	sum:=*sum*+*Y*;

ler_item(*Y*);
Y:=*Y*+*N*;
escrever_item(*Y*);	

← T_3 lê *X* depois da subtração de *N* e lê *Y* antes da adição de *N*; o resultado é um sumário errado (sem *N*).

Recuperação

- Falhas
  - O computador pode falhar
  - Um erro de transação ou sistema
  - Erros locais ou condições de exceção detectadas pela transação
  - Imposição do controle de concorrência
  - Falha de disco
  - Problemas físicos e catástrofes

Log

- Mecanismo utilizado na recuperação
- Log do sistema
  - [inicio_transação, T]
  - [escrever_item, T, X, valor_antigo, valor_novo]
  - [ler_item, T, X]
  - [commit, T]
  - [abort, T]
- *Checkpoint*

Log

T1:read(A);
A : = A-100;
Write(A).

Sendo A com valor inicial R$ 150,00, o log dessa transação é:

< inicio_transacao, T1>
<escreve_item, T1, A, 150,50>
<commit, T1>

Planos de execução

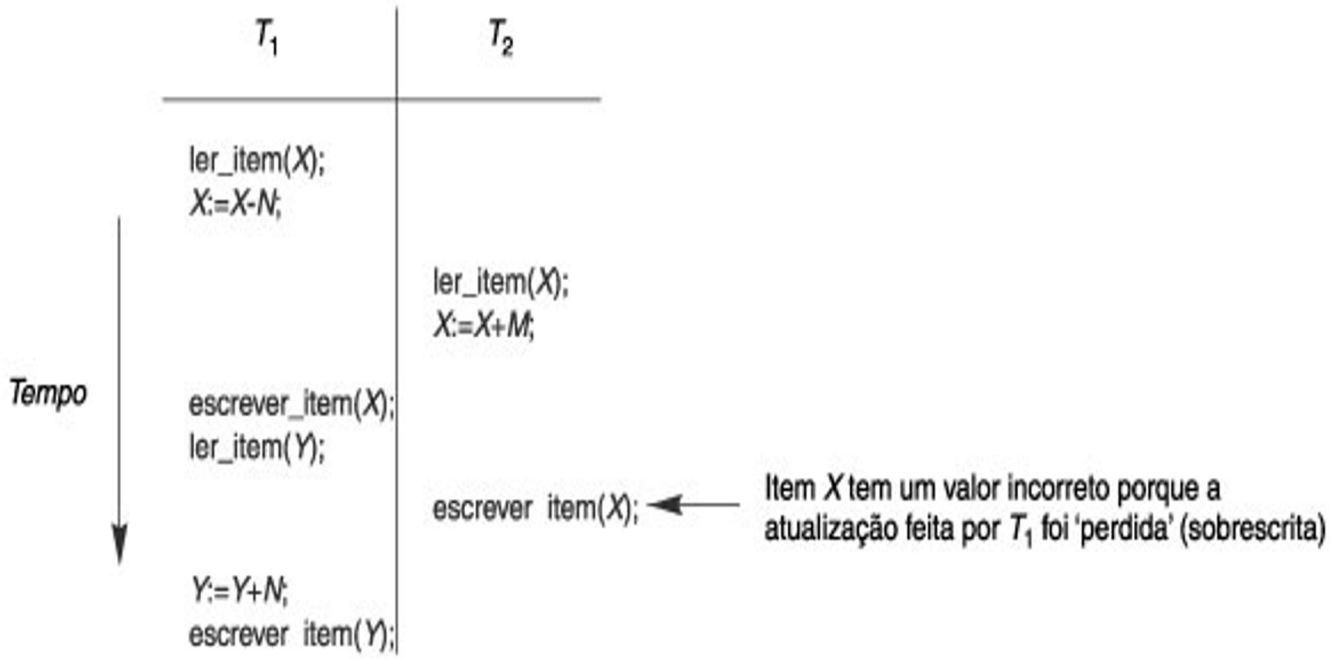
- Processamento de transações concorrentes
  - Plano de execução
    - Restaurabilidade
    - Serialidade
      - Plano serial
      - Plano não-serial

Plano de Execução

- Plano de execução
  - Para cada operação Ti que participe de um plano S, as operações de Ti em S deverão aparecer na mesma ordem em que elas ocorrem em Ti
- Pode haver conflito entre operações se:
  - Pertencerem a diferentes transações
  - Acessarem o mesmo item X
  - Pelo menos uma das operações ser de escrita

Plano de Execução

- S: r1(X); r2(X); w1(X);r1(Y); w2(X); w1(Y)



Plano de Execução - Restaurabilidade

- Plano de execução completo
  - As operações em S são exatamente as operações de T1, T2,..., Tn, tendo um commit ou um abort como última operação de cada transação do plano
  - Para quaisquer pares de operações da mesma transação Ti, sua ordem de aparecimento em S será a mesma que em Ti
  - Para quaisquer duas operações conflitantes, uma das duas precisa aparecer antes da outra no plano
    - Deve haver uma ordenação para que os resultados sejam corretos
    - Nas não-conflitantes isto não é necessário

Plano de Execução - Restaurabilidade

- Se ocorrer uma falha, como um plano pode ser recuperado?

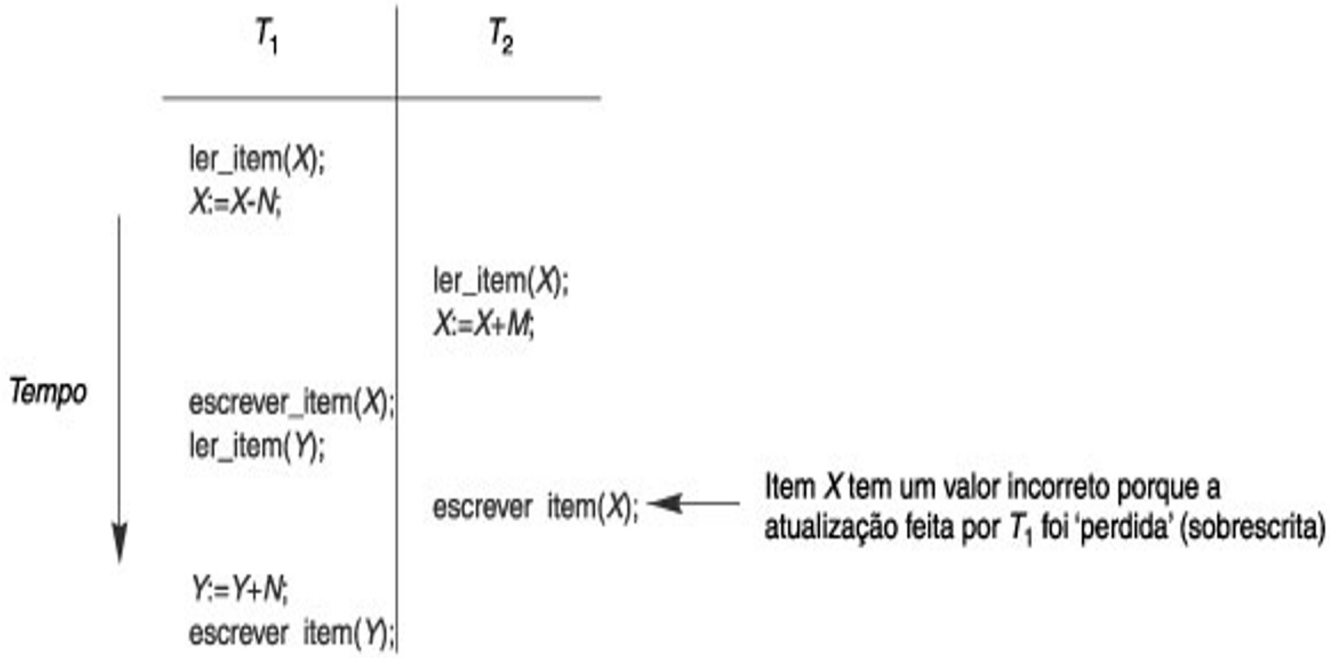
- Plano restaurável
  - Deve garantir que uma vez efetivada uma transação T, nunca mais seja necessário revertê-la

Plano de Execução - Restaurabilidade

- Plano restaurável
  - *Um plano S restaurável é aquele em que, para cada par de transações Ti e Tj em que Tj leia um item de dados previamente escrito por Ti, a operação commit de Ti apareça antes da operação de commit de Tj*
  - Algoritmo para restauração
    - Usando log
  - Restaurável
    - S: r1(X); w1(X); r2(X); r1(Y); w2(X); w1(X); c1; c2

Plano de Execução - Restaurabilidade

- Plano não restaurável
  - S: r1(X); r2(X); w1(X);r1(Y); w2(X); c2; w1(Y); a1



Plano de Execução - Restaurabilidade

- Plano restaurável
  - Reversão livre de cascata
    - Cada transação do plano lê somente itens que já foram gravados por transações efetivadas
    - Ti lê um item gravado por Tj: a operação de leitura de Ti vem depois da operação commit de Tj
  - Reversão que não é livre de cascata

Plano de Execução - Restaurabilidade

- Plano restrito
  - Uma transação não pode nem ler e nem gravar um item X até que a última transação do plano que grave X tenha sido efetivada ou abortada
  - Plano não-restrito
    - Sa: w1(X,5); w2(X,8); a1
  - Plano restrito
    - Sa: w1(X,5); c1; w2(X,8);

Plano de Execução - Serialidade

- Quando um plano intercalado é um plano serializável?
- Plano serial
  - Não existe intercalação entre as operações das transações
  - Limitam a concorrência
- Plano Não-serial
  - Apenas uma transação ativa por vez
  - Serializável: equivalente ao serial
  - Conflito serialidade

(a)

T_1	T_2
ler_item(X);	
X:X–N;	
escrever_item(X)	
ler_item(Y)	
Y:=Y+N;	
escrever_item(Y)	
	ler_item(X);
	X:=X+M;
	escrever_item(X);

Tempo

Plano *A*

(b)

T_1	T_2
	ler item(X);
	X:=X+M;
	escrever_item(X);
ler_item(X);	
X:=X–N;	
escrever_item(X)	
ler_item(Y)	
Y:=Y+N;	
escrever_item(Y)	

Tempo

Plano *B*

(c)

T_1	T_2
ler_item(X);	
X:X–N;	
	ler_item(X);
	X:=X+M;
escrever_item(X)	
ler_item(Y)	
	escrever_item(X);
Y:=Y+N;	
escrever_item(Y)	

Tempo

Plano *C*

T_1	T_2
ler_item(X);	
X:=X–N;	
escrever_item(X)	
	ler_item(X);
	X:=X+M;
	escrever_item(X);
ler_item(Y)	
Y:=Y+N;	
escrever_item(Y)	

Plano *D*

Plano de Execução - Serialidade

- Conflito serialidade
  - Haverá conflito quando houver operação de escrita
  - Duas operações em um plano estarão em conflito se elas pertencerem a transações diferentes, acessarem o mesmo item do banco de dados e pelo menos uma das duas for uma operação escrever_item
  - Um plano S é conflito serializável se ele for (conflito) equivalente a uma plano serial S'

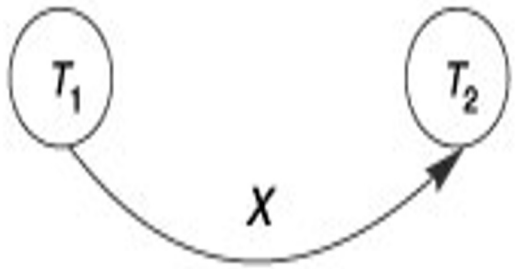
Plano de Execução – Conflito Serialidade

- Algoritmo para testar conflito na serialidade de um plano S
  - Criar um nó rotulado Ti no grafo de precedência para cada transação Ti participante do plano S
  - Criar uma seta (Ti -> Tj) no grafo de precedência para cada caso em S em que
    - Ti ler_item(X) antes que Tj escrever_item(X)
    - Ti escrever_item(X) antes que Tj ler_item(X)
    - Ti escrever_item(X) antes que Tj escrever_item(X)
  - Seta de Ti para Tj indica deTi precisa ser executada antes de Tj
  - O plano S será serializável se, e apenas se, o grafo de precedência não contiver ciclos
    - Ciclo: a sequência começa e termina no mesmo nó

Plano de Execução – Conflito Serialidade

T_1	T_2
ler_item(X);	
X:X–N;	
escrever_item(X)	
ler_item(Y)	
Y:=Y+N;	
escrever_item(Y)	
	ler_item(X);
	X:=X+M;
	escrever_item(X);

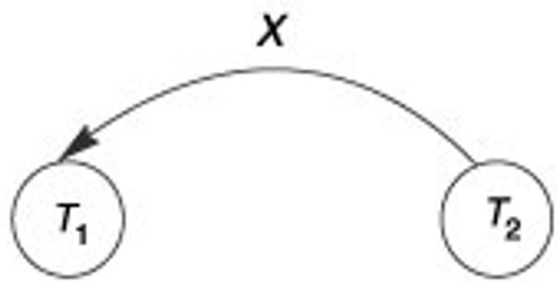
Tempo



Plano de Execução – Conflito Serialidade

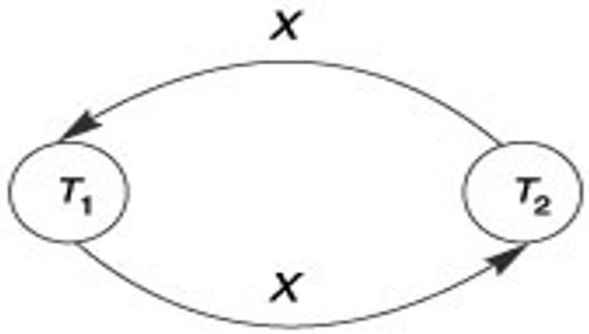
T_1	T_2
	ler_item(X);
	X:=X+M;
	escrever_item(X);
ler_item(X);	
X:=X–N;	
escrever_item(X)	
ler_item(Y)	
Y:=Y+N;	
escrever_item(Y)	

Tempo



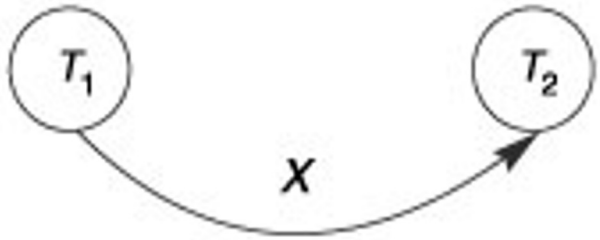
Plano de Execução – Conflito Serialidade

	T_1	T_2
	ler_item(X);	
	X:X–N;	
		ler_item(X);
		X:=X+M;
Tempo	escrever_item(X)	
	ler_item(Y)	
		escrever_item(X);
	Y:=Y+N;	
	escrever_item(Y)	

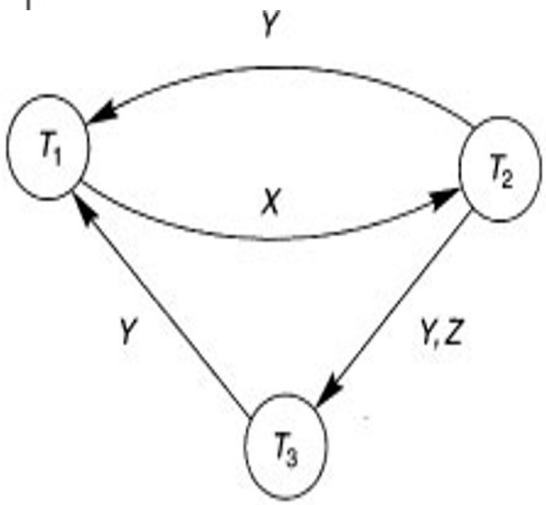


Plano de Execução – Conflito Serialidade

T_1	T_2
ler_item(X);	
X:=X–N;	
escrever_item(X)	
	ler_item(X);
	X:=X+M;
	escrever_item(X);
ler_item(Y)	
Y:=Y+N;	
escrever_item(Y)	



	transação T_1	transação T_2	transação T_3
		ler_item (Z) ler_item (Y) escrever_item (Y);	
			ler_item (Y) ler_item (Z)
Tempo ↓	ler_item (X) escrever_item (X);		escrever_item (Y); escrever_item (Z);
		ler_item (X);	
	ler_item (Y) escrever_item (Y);		
		escrever item (X);	



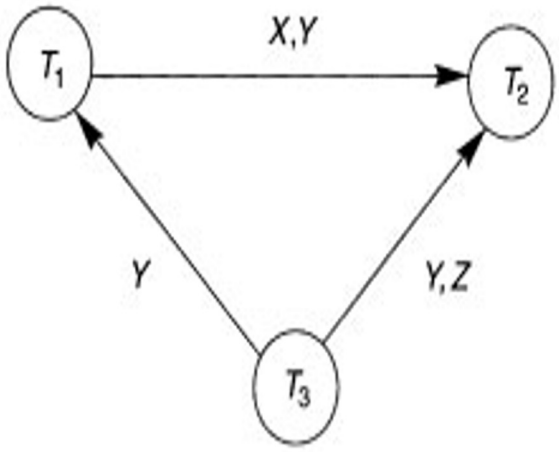
Planos seriais equivalentes

Nenhuma

Razão

ciclo $X(T_1 \rightarrow T_2)$, $Y(T_2 \rightarrow T_1)$
ciclo $X(T_1 \rightarrow T_2)$, $YZ(T_2 \rightarrow T_3)$, $Y(T_3 \rightarrow T_1)$

	transação T_1	transação T_2	transação T_3
			ler_item (Y) ler_item (Z)
Tempo ↓	ler_item (X) escrever_item (X);		escrever_item (Y); escrever_item (Z);
		ler_item (Z)	
	ler_item (Y) escrever_item (Y);		
		ler_item (Y) escrever_item (Y); ler_item (X); escrever_item (X);	



Planos seriais equivalentes

$T_3 \rightarrow T_1 \rightarrow T_2$

Bibliografia utilizada

- Sistemas de Banco de Dados. Elsmari e Navathe. Pearson-Addison Wesley, Quarta Edição (Capitulo 17).